
C U R S O D E A D M I N I S T R A C I Ó N D E K A F K A

LABORATORIO 01

Encender Kafka desde cero

*Guía para hacerlo por tu cuenta,
paso a paso*

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a tener **un servidor de Kafka funcionando**, levantado por ti desde cero, con su primer tópico creado.

Y más importante que eso: **vas a entender por qué cada paso es necesario**, en vez de haber copiado comandos.

LO QUE ESTE LABORATORIO DEMUESTRA

*Un servidor de Kafka no arranca sin una identidad grabada en su disco.
Y esa identidad se decide una sola vez.*

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo	Que el ícono de la ballena esté activo
Git Bash abierto	Es la terminal que usamos en todo el curso
El repositorio del curso	Debe estar en <code>C:\KAFKA\curso-kafka-sunat</code>

ANTES DE TOCAR NADA · DOS COSAS

Uno. Actualiza el repositorio, porque el material se corrige durante el curso:

```
cd /c/KAFKA/curso-kafka-sunat
git pull
```

Dos. Comprueba que no tengas otro laboratorio levantado, porque ocuparía los mismos puertos:

```
docker ps --filter name=kafka-broker
```

Si aparece algo, bájalo antes con `docker compose -p novatech-labNN down -v`, reemplazando NN por el número del laboratorio que aparezca.

El caso

NovaTech Logistics coordina envíos, flota y entregas en tiempo real, y está montando su plataforma de eventos sobre Kafka.

Su director de tecnología tiene una regla: **no deja operar a nadie que no sepa encender la plataforma desde cero**. Eso es lo que vas a hacer hoy.

Mirar qué trae Kafka por dentro

Qué vamos a hacer

Antes de encender nada, vamos a **abrir la caja**. Kafka viene empaquetado en lo que se llama una *imagen*, y adentro trae los programas que vas a usar todo el curso.

PARA ENTENDERLO MEJOR

Imagina que la empresa compra un camión nuevo. **Antes de manejarlo, abres el capó y miras qué motor trae.** No lo enciendes todavía: solo quieres saber con qué estás trabajando.

CONCEPTO · IMAGEN

Una **imagen** es un paquete que trae un programa y todo lo que necesita para funcionar. Cuando la ejecutas, se convierte en un **contenedor**, que es una copia viva de ese paquete, corriendo.

El comando

```
cd Capitulo_2/lab-01-inicializacion-kraft  
kafka-cli/inspect-image.sh
```

cd Capitulo_2/lab-01-inicializacion-kraft — entra a la carpeta del laboratorio

kafka-cli/inspect-image.sh — un script del curso que abre la imagen de Kafka y te muestra qué contiene. **No levanta ningún servidor**

LO QUE VAS A VER

Primero una caja explicativa que anuncia los cuatro pasos que va a dar.
Después:

```
[1/4] la imagen y cuánto pesa  
[2/4] los programas kafka-* que trae adentro  
[3/4] la versión de Java sobre la que corre  
[4/4] un archivo de configuración de ejemplo
```

VAS BIEN SI...

Viste los cuatro pasos numerados y terminó sin errores. **Si el paso [1/4] se demora varios minutos, es porque está descargando la imagen por primera vez.** Son unos 800 MB y solo pasa una vez.

LO QUE HAY QUE RETENER

Esos programas `kafka-topics`, `kafka-storage` y los demás **son los mismos que vas a usar en un servidor real.**

La diferencia es que aquí los invocamos a través de Docker, y en producción están instalados directamente en el sistema. **Los comandos no cambian.**

Leer la configuración de fábrica

Qué vamos a hacer

La imagen trae un archivo de configuración de ejemplo. **Ese no es el que va a usar nuestro servidor.** El nuestro lo armamos nosotros. Pero mirarlo sirve para ver qué decisiones trae Kafka por defecto.

PARA ENTENDERLO MEJOR

Es el **manual de fábrica** que viene en la guantera. No es cómo tú vas a manejar el camión, pero conviene leerlo antes.

El comando

```
MSYS_NO_PATHCONV=1 docker run --rm confluentinc/cp-kafka:8.2.0 \
  cat /etc/kafka/server.properties | head -50
```

MSYS_NO_PATHCONV=1 — solo hace falta en Git Bash sobre Windows. Le dice que **no toque las rutas de este comando**. Sin esto, Windows convertiría `/etc/kafka/...` —que es una ruta dentro del contenedor— en una ruta de tu disco C:, y fallaría

docker run — crea un contenedor nuevo a partir de la imagen

--rm — lo borra apenas termina. Es de usar y tirar

cat /etc/kafka/server.properties — el comando que se ejecuta adentro. `cat` muestra el contenido de un archivo

| head -50 — la tubería `|` toma la salida y se la pasa al siguiente comando. `head -50` se queda con las primeras cincuenta líneas

Tres preguntas para responder mirando la salida

Pregunta	Dónde está la respuesta
¿Qué roles trae este servidor por defecto?	La línea <code>process.roles</code>
¿En qué puerto escucha?	La línea <code>listeners</code>
¿Dónde guarda los datos?	No aparece. Sigue leyendo

LAS DOS PRIMERAS RESPUESTAS

```
process.roles=broker,controller
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
```

QUÉ SIGNIFICAN

`process.roles=broker,controller` — este servidor cumple dos papeles a la vez. Como **broker** recibe y entrega mensajes; como **controller** coordina el clúster. En instalaciones grandes esos roles se separan.

`listeners` — las puertas por donde escucha. El 9092 para clientes, el 9093 para la coordinación interna.

La tercera pregunta necesita otro comando

La propiedad que buscas está en la línea 73, y el `head -50` la dejó fuera. **En vez de leer el archivo entero, pregúntale directamente por ella:**

```
docker run --rm confluentinc/cp-kafka:8.2.0 \
  sh -c "grep '^log.dirs' /etc/kafka/server.properties"
```

`sh -c "..."` — abre una consola dentro del contenedor para ejecutar lo que va entre comillas. **Hace falta porque el `grep` tiene que correr ahí dentro**

`grep '^log.dirs'` — busca las líneas que **empiezan** por `log.dirs`. El símbolo `^` significa «principio de línea»; sin él saldrían también los comentarios que la mencionan de paso

LO QUE VAS A VER

```
log.dirs=/tmp/kraft-combined-logs
```

AQUÍ HAY DOS COSAS IMPORTANTES

La primera es una trampa de nombre. En Kafka, «log» no significa los mensajes que escupe un programa. **El log ES la base de datos de Kafka**, el registro donde queda escrito cada mensaje. `log.dirs` dice dónde vive ese registro en el disco.

La segunda es dónde lo pone el ejemplo: en `/tmp`, que en Linux es una carpeta que se vacía al reiniciar el sistema. **Un servidor que guarda sus datos ahí los pierde en el próximo reinicio.** Por eso este archivo es un ejemplo y no una configuración de producción.

VAS BIEN SI...

Respondiste las tres preguntas y entiendes por qué la tercera necesitó un comando aparte.

Generar la identidad del clúster

Qué vamos a hacer

Todo clúster de Kafka nace con un **identificador único**. Se genera una vez, se graba en el disco de cada servidor, y no se cambia nunca más.

PARA ENTENDERLO MEJOR

Es el **nombre de la empresa pintado en la puerta del camión**. Todos los camiones de la flota llevan el mismo. Si uno trae otro nombre pintado, no es de esta flota.

El comando

```
kafka-cli/generate-cluster-id.sh
```

kafka-cli/generate-cluster-id.sh — un script del curso. Por dentro ejecuta `kafka-storage random-uuid`, que genera un identificador nuevo. **No toca ningún clúster ni ningún disco**

LO QUE VAS A VER

Una caja explicativa, y dentro de ella una línea como esta:

```
hisDgG37QLqymPQNOoKg4g
```

COPIA ESE VALOR AHORA

Lo vas a necesitar en el paso siguiente. Pégallo en un bloc de notas.

Y no uses el del ejemplo ni el de un compañero: usa el tuyo. Cada ejecución genera uno distinto y son 22 caracteres irrepetibles.

POR QUÉ KAFKA ES TAN ESTRICTO CON ESTO

Cuando un servidor arranca, **lo primero que hace es leer su identidad en el disco**. Si no la encuentra, se niega a partir. Si encuentra una distinta a la que le dijeron, muere con un error llamado `InconsistentClusterIdException`.

¿Por qué tanto rigor? Porque el peor desastre posible es un servidor escribiendo datos de un clúster en el disco de otro. Eso corrompe la información en silencio. **Kafka prefiere no arrancar antes que arriesgarse.**

Preparar el plano del servidor

Qué vamos a hacer

Necesitamos un archivo que le diga a Docker cómo armar nuestro servidor. Ese archivo se llama `docker-compose.yml`.

El laboratorio trae una versión resuelta. **La copiamos y le cambiamos una sola línea: la identidad que acabas de generar.**

Los comandos

```
cp soluciones/docker-compose-1-broker.SOLUCION.yml mi-cluster/docker-compose.yml
code mi-cluster/docker-compose.yml
```

`cp <origen> <destino>` — copia el plano resuelto a tu carpeta de trabajo. **Este comando no imprime nada: en Unix, el silencio significa que salió bien**

`code <archivo>` — lo abre en VSCode. Si no tienes VSCode, usa `notepad mi-cluster/docker-compose.yml`

El cambio que tienes que hacer

Busca la línea que dice `CLUSTER_ID`, cerca de la línea 22. **Reemplaza el valor entre comillas por el tuyo**, el que generaste en el paso 3. Las comillas se quedan.

DEBE QUEDAR ASÍ, CON TU IDENTIFICADOR

```
CLUSTER_ID: 'hisDgG37QLqymPQN0oKg4g'
```

Guarda con **Ctrl+S**.

Las tres líneas del archivo que vale la pena entender

```
1 · KAFKA_PROCESS_ROLES
```

Dice `'broker,controller'`. **Este servidor va a cumplir los dos papeles a la vez.**

Y fijate en algo que **no** está en el archivo: ZooKeeper. En versiones antiguas de Kafka hacía falta un sistema aparte para coordinar el clúster. **Desde Kafka 4 ya no existe: la coordinación es parte de Kafka.** Eso se llama modo KRaft.

2 · KAFKA_LISTENERS Y KAFKA_ADVERTISED_LISTENERS

Los listeners son las puertas que el servidor abre. Hay tres: una para clientes externos, una para hablar con otros servidores, y una para la coordinación.

Los advertised listeners son la dirección que el servidor le publica a sus clientes para que vuelvan. Si esa dirección está mal, el cliente conecta la primera vez y falla después. **Es el error más común de Kafka en producción**, y lo vas a ver a fondo en el laboratorio 04.

3 · LOS TRES REPLICATION_FACTOR EN 1

El factor de replicación es cuántas copias se guardan de cada dato. Aquí es 1 porque tenemos un solo servidor: no hay dónde poner una segunda copia.

En producción ese valor es 3, siempre. Y en el laboratorio 02, cuando levantes tres servidores, estas líneas cambian.

Encender el servidor

Antes de ejecutar, una pregunta

Nosotros escribimos la identidad en el archivo de configuración. **Pero nadie la grabó todavía en el disco del servidor.**

Entonces, ¿va a arrancar? **Piénsalo antes de seguir leyendo.**

LA RESPUESTA

Sí va a arrancar. **La imagen de Confluent, la primera vez que arranca, si le pasas un `CLUSTER_ID` y el disco está vacío, formatea el almacenamiento sola.**

Pero esa cortesía existe solo la primera vez y solo con el disco vacío. Retén eso: en el desafío final vas a ver qué pasa cuando no se cumple.

Los comandos

```
cd mi-cluster
docker compose up -d
```

cd mi-cluster — entra a la carpeta donde está tu plano

docker compose up -d — crea y levanta todo lo que el plano declara. La `-d` significa *detached*: corre en segundo plano y te devuelve la terminal

LO QUE VAS A VER

```
[+] up 3/3
✓ Network novatech-lab01_default    Created
✓ Volume novatech-lab01_kafka-data  Created
✓ Container kafka-broker            Started
```

Comprueba que está vivo

```
docker ps --filter name=kafka-broker
```

docker ps — lista los contenedores que están corriendo

`--filter name=kafka-broker` — muestra solo los que se llaman así. **En un servidor con muchos contenedores, este filtro es tu mejor amigo**

VAS BIEN SI...

La columna `STATUS` dice `Up` y en `PORTS` aparece `0.0.0.0:9092->9092/tcp`.

SI DICE RESTARTING

Significa que el servidor arranca y se muere en bucle. **Casi siempre es el `CLUSTER_ID` mal pegado.** Un espacio de más, o una comilla que se perdió.

Míralo tú mismo con: `docker logs kafka-broker`

Tómale el pulso

```
cd ..  
docker exec kafka-broker kafka-broker-api-versions \  
  --bootstrap-server kafka-broker:29092
```

`cd ..` — vuelve a la raíz del laboratorio

`docker exec` — ejecuta un comando **dentro** de un contenedor que ya está corriendo

`kafka-broker` — en cuál contenedor

`kafka-broker-api-versions` — un comando de Kafka que pregunta qué operaciones soporta el servidor

`--bootstrap-server kafka-broker:29092` — a qué servidor le habla y por qué puerta. **Usamos el 29092 —la puerta interna— porque con `docker exec` estamos dentro de la red de Docker**

VAS BIEN SI...

Sale **una lista larga** de operaciones. **Aquí lo largo es la buena noticia:** si el servidor estuviera caído verías una sola línea corta con un error.

Fíjate en la primera línea. Dice `id: 1`, que es el número de este servidor, y `isFenced: false`, que significa que está habilitado para atender clientes.

Crear tu primer tópico

Qué vamos a hacer

CONCEPTO · TÓPICO

Un **tópico** es donde se anotan los mensajes, en orden, uno tras otro. Es el registro donde queda escrito cada evento.

Y algo que lo distingue de una cola tradicional: leer un mensaje no lo borra. Queda ahí, y otro sistema puede leerlo después.

El comando

```
docker exec kafka-broker kafka-topics \  
  --bootstrap-server kafka-broker:29092 \  
  --create --topic test-solitario \  
  --partitions 1 --replication-factor 1
```

docker exec kafka-broker — ejecuta dentro del contenedor del servidor

kafka-topics — la herramienta que administra tópicos. **Es el comando que más vas a usar en tu vida de administrador**

--bootstrap-server kafka-broker:29092 — a qué servidor le habla y por qué puerta

--create — la acción: crear

--topic test-solitario — el nombre del tópico

--partitions 1 — **en cuántos pedazos se divide.** Las particiones permiten que varios servidores trabajen en paralelo

--replication-factor 1 — cuántas copias se guardan. **Con un solo servidor no puede ser más de 1**

LO QUE VAS A VER

```
Created topic test-solitario.
```

Comprueba que existe

```
docker exec kafka-broker kafka-topics \  
  --bootstrap-server kafka-broker:29092 \  
  --list
```

--list — en vez de crear, lista los tópicos que existen

VAS BIEN SI...

Sale **una sola línea**: `test-solitario`.

Y fíjate en lo que NO aparece: Kafka tiene tópicos internos propios, pero nacen cuando alguien los necesita. Tu clúster es tan nuevo que todavía no ha necesitado ninguno.

Ahora provoca un error a propósito

Intenta crear un tópico pidiendo **tres copias**, cuando solo tienes un servidor:

```
docker exec kafka-broker kafka-topics \  
  --bootstrap-server kafka-broker:29092 \  
  --create --topic test-triple \  
  --partitions 1 --replication-factor 3
```

--replication-factor 3 — **aquí está la trampa**. Pide tres copias de cada partición, y solo hay un servidor donde ponerlas

EL ERROR QUE VAS A VER

```
Error while executing topic command : Unable to replicate the partition 3 time(s):  
The target replication factor of 3 cannot be reached because only 1 broker(s) are registered.
```

DOS COSAS QUE APRENDER DE ESTE ERROR

La primera: Kafka no dice «error 47». **Te dice exactamente qué pediste y qué tiene.**

Acostúmbrate a leer los errores de Kafka completos, porque casi siempre traen la respuesta adentro.

La segunda: debajo del mensaje hay varias líneas que empiezan con `at java.base...`. **Esa es la traza interna de Java y no te dice nada útil.** La información está en la primera línea.

Y el remate: si vuelves a listar, `test-triple` no existe. **Kafka valida antes de crear: o nace completo, o no nace.**

El desafío · cambiar la identidad

De qué se trata

Al principio de esta guía se afirmó que **la identidad del clúster se graba en el disco y no se cambia**. Ahora vas a comprobarlo tú.

Vas a **destruir el servidor completo, incluido su disco**, y hacerlo renacer con una identidad nueva.

Destruye todo

```
cd mi-cluster
docker compose down -v
cd ..
```

`docker compose down` — apaga y elimina los contenedores del plano

`-v` — **esta es la letra peligrosa. Borra también los volúmenes**, o sea el disco con la identidad grabada

LA DIFERENCIA ENTRE DOWN Y DOWN -V

`down` a secas apaga los contenedores pero **conserva el disco**. Al volver a levantar, los datos siguen ahí.

`down -v` borra el disco también. **Es la decisión que más vas a tomar en este curso**, y en producción es la diferencia entre reiniciar un servicio y perder los datos.

VAS BIEN SI...

Salen tres líneas con `Removed` : el contenedor, el volumen y la red. **Y las tres llevan el prefijo `novatech-lab01`** . El comando solo tocó lo suyo.

Genera una identidad nueva

```
kafka-cli/generate-cluster-id.sh
```

Cópiala. **Compárala con la anterior: son completamente distintas.**

Cámbiala en el plano y enciende

```
code mi-cluster/docker-compose.yml
```

Reemplaza el `CLUSTER_ID` por el nuevo y guarda. Después:

```
cd mi-cluster
docker compose up -d
cd ..
```

POR QUÉ ESTO FUNCIONA

Borraste el disco con la `-v`, así que el servidor arranca con el almacenamiento vacío. **Y con el disco vacío, la cortesía de la primera vez opera otra vez:** formatea solo, con la identidad nueva.

Y ahora ábrele el disco

Espera unos diez segundos y ejecuta:

```
bin/verify-storage.sh kafka-broker
```

`bin/verify-storage.sh` — un script del curso que le pide al servidor la información grabada en su disco

`kafka-broker` — el nombre del contenedor, y es obligatorio. En este laboratorio se llama así, sin número. Desde el laboratorio 02 serán `kafka-broker-1`, `-2` y `-3`

LO QUE VAS A VER

```
Found log directory:
  /var/lib/kafka/data
Found metadata: {cluster.id=TU_IDENTIDAD_NUEVA,
  directory.id=HR1Dn3R4w9C9Do8T3lvRLQ, node.id=1, version=1}
```

LO LOGRASTE SI...

El `cluster.id` que aparece **es el que generaste hace tres minutos**, no el anterior.

Eso cierra el círculo: **generaste una identidad, la escribiste en el plano, el servidor la grabó en su disco, y la lee de vuelta.**

LOS TRES VALORES QUE APARECEN

`cluster.id` — la identidad del clúster. **Todos los servidores de un mismo clúster llevan exactamente esta misma.**

`node.id` — quién es este servidor dentro del clúster. Único por servidor.

`directory.id` — identifica este directorio de datos en particular. **Es distinto en cada servidor y no se copia entre ellos.**

Lo que aprendiste

- 1 Un servidor de Kafka no arranca sin identidad grabada en su disco.**
Y si encuentra una distinta a la esperada, se niega a partir. Prefiere no arrancar antes que mezclar datos de dos clústeres.
- 2 El factor de replicación no puede superar el número de servidores.**
Con uno, el máximo es 1. Con tres, puedes pedir 3, y en producción es lo que corresponde.
- 3 Los datos nunca van en `/tmp`.**
Esa carpeta se vacía al reiniciar el sistema. En producción los datos van en un disco dedicado, con espacio monitoreado.
- 4 La diferencia entre `down` y `down -v` es la diferencia entre apagar y destruir.**
Ten claro cuál estás ejecutando, siempre.

Para profundizar

Si quieres ir más allá, prueba esto por tu cuenta:

Ejercicio	Qué vas a descubrir
Levanta el servidor, apágalo con <code>down</code> sin <code>-v</code> , cambia el <code>CLUSTER_ID</code> en el plano y vuelve a levantar	El error <code>InconsistentClusterIdException</code> en vivo. Léelo completo
Crea un tópico con <code>--partitions 3</code> y descríbelo con <code>--describe</code>	Cómo Kafka reparte las particiones cuando solo hay un servidor
Mira los archivos del disco con <code>MSYS_NO_PATHCONV=1 docker exec kafka-broker ls /var/lib/kafka/data</code>	Que cada partición es una carpeta, y adentro hay archivos <code>.log</code>
Corre <code>docker logs kafka-broker</code> completo	Todo lo que Kafka hace al arrancar, incluido el formateo automático

Antes de cerrar

DEJA EL TERRENO LIMPIO

Si vas a hacer otro laboratorio, **baja este primero** o los puertos van a estar ocupados:

```
cd mi-cluster  
docker compose down -v  
cd ..
```

Y comprueba con `docker ps --filter name=kafka-broker` que no quede nada.